Problem Statement

In computer science, a double-ended queue (dequeue, often abbreviated to deque, pronounced deck) is an abstract data type that generalizes a queue, for which elements can be added to or removed from either the front (head) or back (tail).

Deque interfaces can be implemented using various types of collections such as LinkedList or ArrayDeque classes. For example, deque can be declared as:

Deque deque = new LinkedList<>();

or

Deque deque = new ArrayDeque<>();

You can find more details about Deque here.

In this problem, you are given integers. You need to find the maximum number of unique integers among all the possible contiguous subarrays of size .

Note: Time limit is second for this problem.

Input Format

The first line of input contains two integers and : representing the total number of integers and the size of the subarray, respectively. The next line contains space separated integers.

Constraints

The numbers in the array will range between .

Output Format

Print the *maximum* number of unique integers among all possible contiguous subarrays of size .

Sample Input

6 3

5 3 5 2 3 2

Sample Output

3

Explanation

In the sample testcase, there are 4 subarrays of contiguous numbers.

- Has unique numbers.

- Has unique numbers.

- Has unique numbers.

- Has unique numbers.

In these subarrays, there are unique numbers, respectively. The maximum amount of unique numbers among all possible contiguous subarrays is .

Program:

```
import java.util.*;


public class Solution {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);


      int n = sc.nextInt(); // total number of integers
      int m = sc.nextInt(); // size of subarray


      int[] arr = new int[n];
      for (int i = 0; i < n; i++) {
         arr[i] = sc.nextInt();
      }


      // Deque to represent sliding window
      Deque<Integer> window = new ArrayDeque<>();
      // HashMap to store frequency of elements in current window
      Map<Integer, Integer> freqMap = new HashMap<>();


      int maxUnique = 0;


      for (int i = 0; i < n; i++) {
         int num = arr[i];
         window.addLast(num);
         freqMap.put(num, freqMap.getOrDefault(num, 0) + 1);


         // Maintain window size = m
         if (window.size() > m) {
```

```
            int removed = window.removeFirst();
            freqMap.put(removed, freqMap.get(removed) - 1);
            if (freqMap.get(removed) == 0) {
                freqMap.remove(removed);
            }
        }

        // When window size is exactly m, check unique count
        if (window.size() == m) {
            maxUnique = Math.max(maxUnique, freqMap.size());
        }
    }

    System.out.println(maxUnique);

    sc.close();
  }
}
```

Sample Input

6 3

5 3 5 2 3 2

Sample Output

3